TM261 – Agent-Led Test Management: Hands-on Lab with Joule in SAP Cloud ALM

Presenter Script – Full Session

Session	TM261 – Agent-Led Test Management: Hands-on Lab with Joule in SAP Cloud ALM
Date	Thursday, 29 October 2026
Location	SAP TechEd 2026, Berlin
Presenters	Christina Pickhardt · Niclas Weidner · Tobias Keller
Scenario	Sales Order Processing with Customer Down Payment (BKJ)

> This file is the compiled presenter script. Source of truth lives in the individual exercise READMEs under . Images are referenced relative to the repo root.

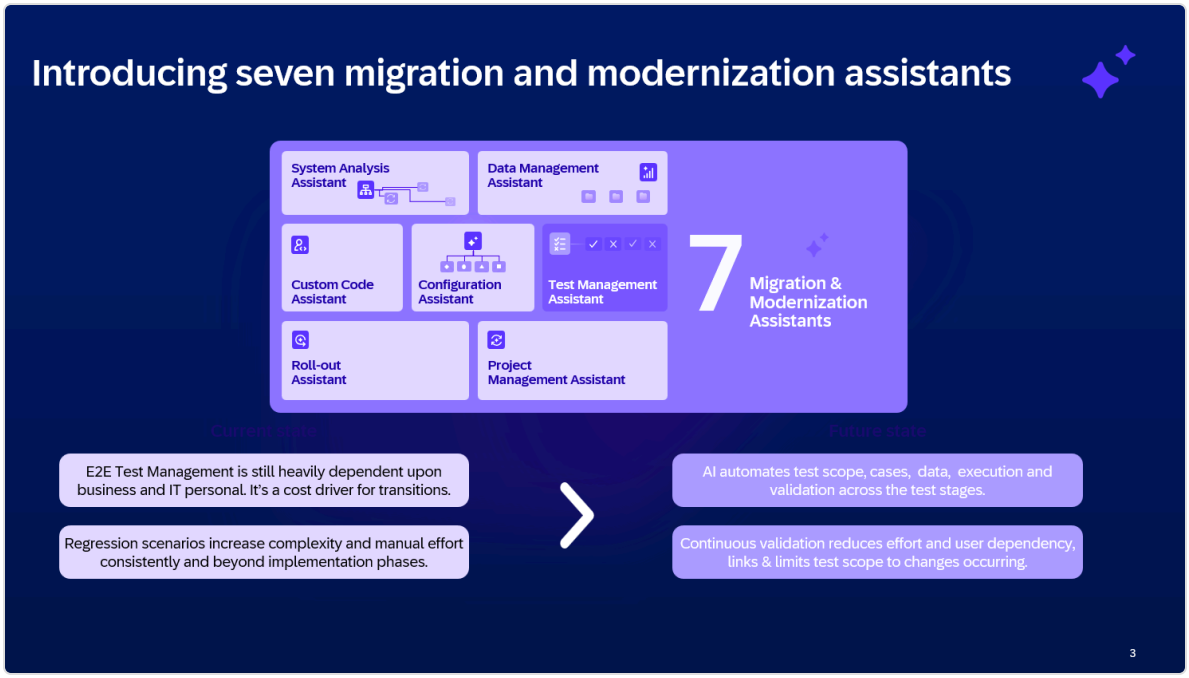
Agenda

#	Section	Topic
–	Introduction	SAP Migration & Modernization Assistants: context
–	Introduction	Test Management Assistant: agent architecture overview
0	Getting Started	Log In & Orient Yourself · Steps 1–4C
1	Generate Requirements	Turn an interview transcript into requirements with AI · Step 5
2	Scope with AI	Scoping agent; compare with manual scoping · Steps 6–10
3	Plan with AI	Planning agent; HITL Gate 2; review · Steps 11–14
4	Generate Test Cases	TC generation agent; quality review; parameters · Steps 15–18
5	Generate Test Data	Data agent; classify & explore; dependencies · Steps 19–23
6	Recap & Outlook	Time saved; roadmap; key takeaways · Steps 24–26

Introduction – SAP Migration & Modernization Assistants

> **Presenter narration:** Before we jump into the hands-on, a quick word on context, because what you're about to use is not an isolated feature.

SAP Cloud ALM Joule is the home of the **Test Management Assistant**: one of **seven** AI-powered assistants SAP is building for Migration and Modernization scenarios:



Assistant	
System Analysis Assistant	
Data Management Assistant	
Custom Code Assistant	
Configuration Assistant	
Test Management Assistant	← you are here
Roll-out Assistant	
Project Management Assistant	

Why it matters for test management specifically:

Current State	Future State
E2E Test Management is still heavily dependent upon business and IT personal; it's a cost driver	AI automates test scope, cases, data, execution and validation across the test

Current State	Future State
for transitions.	stages.
Regression scenarios increase complexity and manual effort consistently and beyond implementation phases.	Continuous validation reduces effort and user dependency, links & limits test scope to changes occurring.

> The Test Management Assistant is the one you're working with today. It sits at the quality gate of every migration, and as you'll see, it's where AI can compress days of manual work into minutes of guided review.

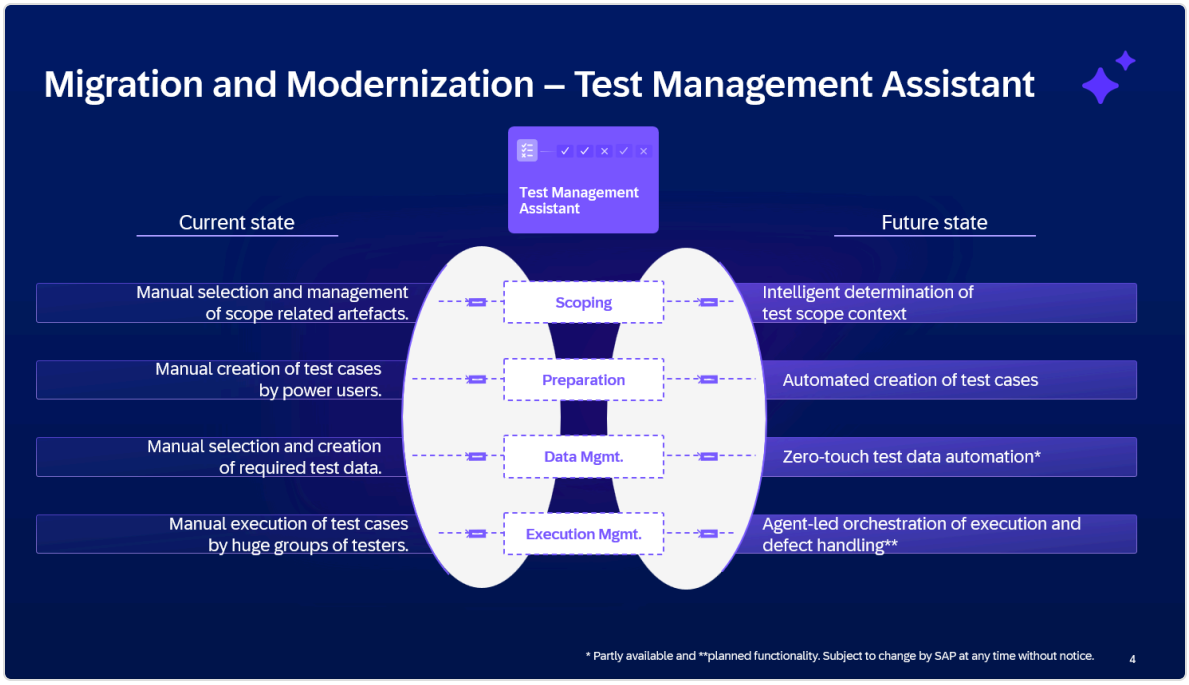
Introduction – Test Management Assistant: Agent Architecture

> **Presenter narration:** The Test Management Assistant is not a single AI feature; it's a pipeline of four specialized agents, each transforming manual work into an AI-driven capability.

The challenge each agent addresses:

Current State (Manual)	Agent	Future State (AI-driven)
Manual selection and management of scope-related artefacts	**Scoping**	Intelligent determination of test scope context
Manual creation of test cases by power users	**Preparation**	Automated creation of test cases
Manual selection and creation of required test data	**Data Mgmt.**	Zero-touch test data automation *(partly available)*
Manual execution of test cases by huge groups of testers	**Execution Mgmt.**	Agent-led orchestration of execution and defect handling *(planned)*

> **Today's hands-on covers the first three agents**: Scoping, Preparation, and Data Management. Execution Management is on the roadmap.
>
> ** Partly available and **planned functionality. Subject to change by SAP at any time without notice.*


Migration and Modernization – Test Management Assistant
Test Management Assistant
Current state
Future state
Manual selection and management of scope related artefacts.
Scoping
Intelligent determination of test scope context
Manual creation of test cases by power users.
Preparation
Automated creation of test cases
Manual selection and creation of required test data.
Data Mgmt.
Zero-touch test data automation*
Manual execution of test cases by huge groups of testers.
Execution Mgmt.
Agent-led orchestration of execution and defect handling**
* Partly available and **planned functionality. Subject to change by SAP at any time without notice.
4

Getting Started – Log In & Orient Yourself

Session: TM261 – Agent-Led Test Management: Hands-on Lab with Joule in SAP Cloud ALM
Date: Thursday, 29 October 2026 · SAP TechEd 2026, Berlin
Presenters: Christina Pickhardt · Niclas Weidner · Tobias Keller
Scenario: Sales Order Processing with Customer Down Payment (BKJ)

Your Story Today

You are a **Test Business Process Expert** at a company that just activated SAP S/4HANA Scope Item BKJ: Sales Order Processing with Customer Down Payment. Your task: **prepare test coverage for the new process** from scratch.

In this hands-on you'll work with AI agents in SAP Cloud ALM Test Management to speed up every stage, from requirements to test data, dramatically reducing the manual effort that would normally take days.

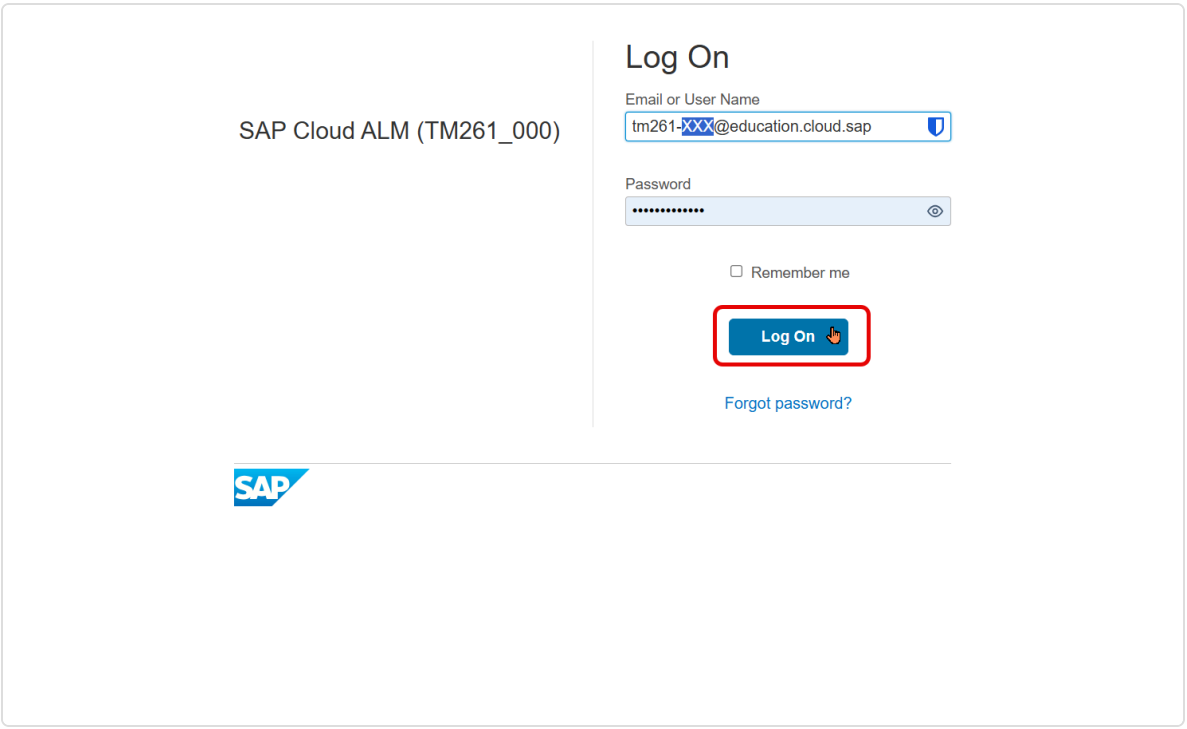
Step 1 – Log In to SAP Cloud ALM

You are a business and process expert at a large multinational corporation. It's the start of your work day, and you have an important task on your agenda: preparing test coverage for a newly activated SAP process.

Open your browser. The SAP Cloud ALM login page is already loaded.

> **Your credentials:**
>
> | Field | Value |
> |---|---|
> | User | `tm261-XXX@education.cloud.sap` |
> | Password | `Acce$$teched26` |
>
> Replace **XXX** with your **workstation number** (shown on your desk). This number is yours for the entire session; it identifies your personal data and work in the shared system.

1. Confirm the tenant name on the left reads **SAP Cloud ALM (TM261_000)**: this is the shared demo system for this session.
2. Enter your username and password, then click **Log On**.

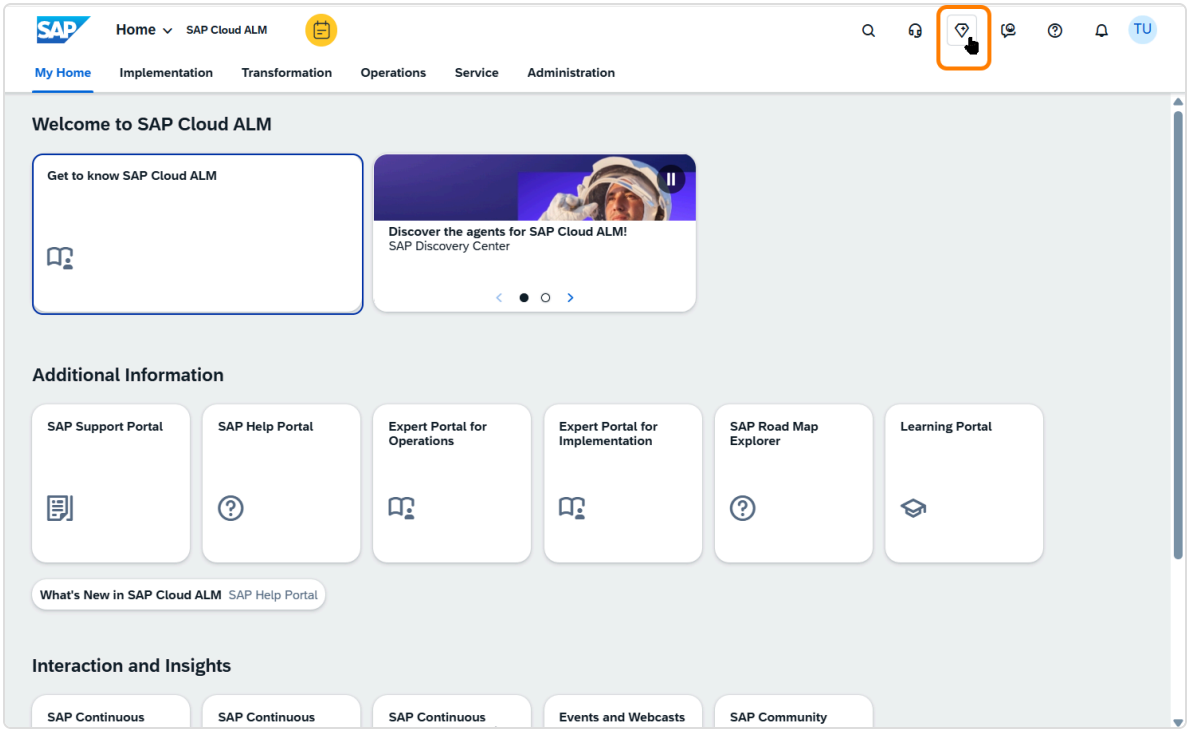


Step 2 – Open Joule

After logging in you land on the SAP Cloud ALM home page. Time to call your most valuable tool: **Joule**.

Joule is a chat-style AI interface built into SAP Cloud ALM. You'll use it throughout this session to interact with AI agents, ask questions, and drive the test preparation workflow; all in natural language.

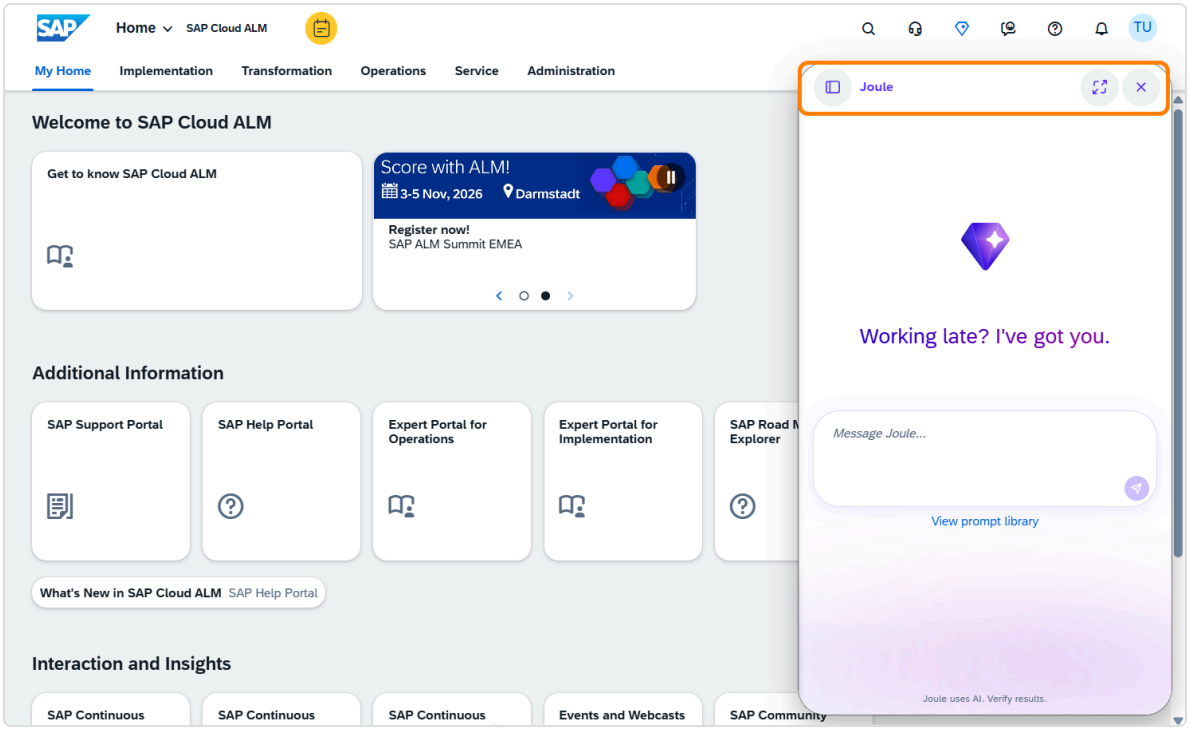
1. Look for the **diamond icon** (◇) in the upper-right area of the screen and click it to open Joule.



1. Joule opens as a panel that slides over the application screen; it's always available, regardless of where you are in SAP Cloud ALM.

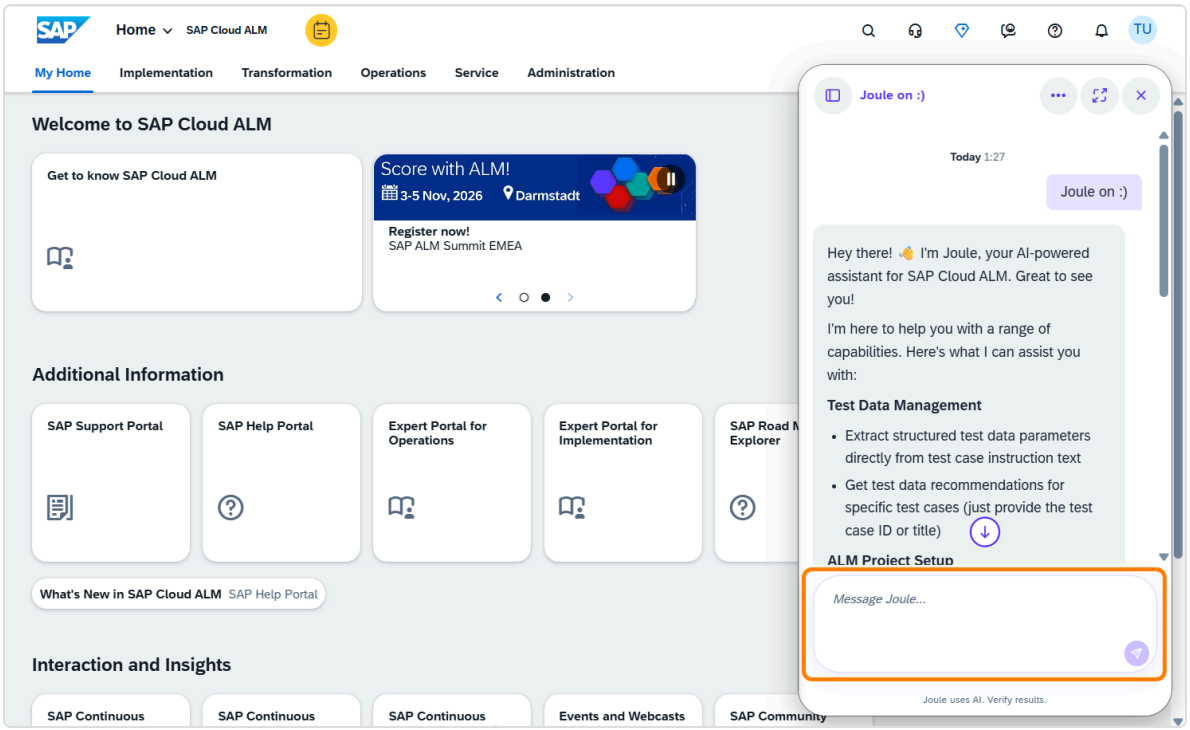
The panel header gives you three controls:
- **Sidebar icon**: manage multiple concurrent conversations (you can run several Joule chats in parallel, e.g. one per topic or agent)
- **Expand icon**: open Joule in full-screen mode
- **Close (×)**: close the panel

> **Good to know:** Joule is stateful. Close the panel, navigate elsewhere, reopen it; your conversation is still there exactly as you left it.



1. Wake up Joule with a simple message in the input box at the bottom; just type naturally, for example: **"Joule on :)"** or any greeting.

Joule responds with an introduction of its capabilities. Scroll through its response; you'll see areas like **Test Data Management**, **Test Scoping**, **Test Planning**, and **ALM Project Setup**. These are exactly the capabilities you'll use today.



Step 3 – Jump In Early *(Optional)*

> **Optional step**: try this if you want to see what happens when you skip straight to test case generation without proper setup.

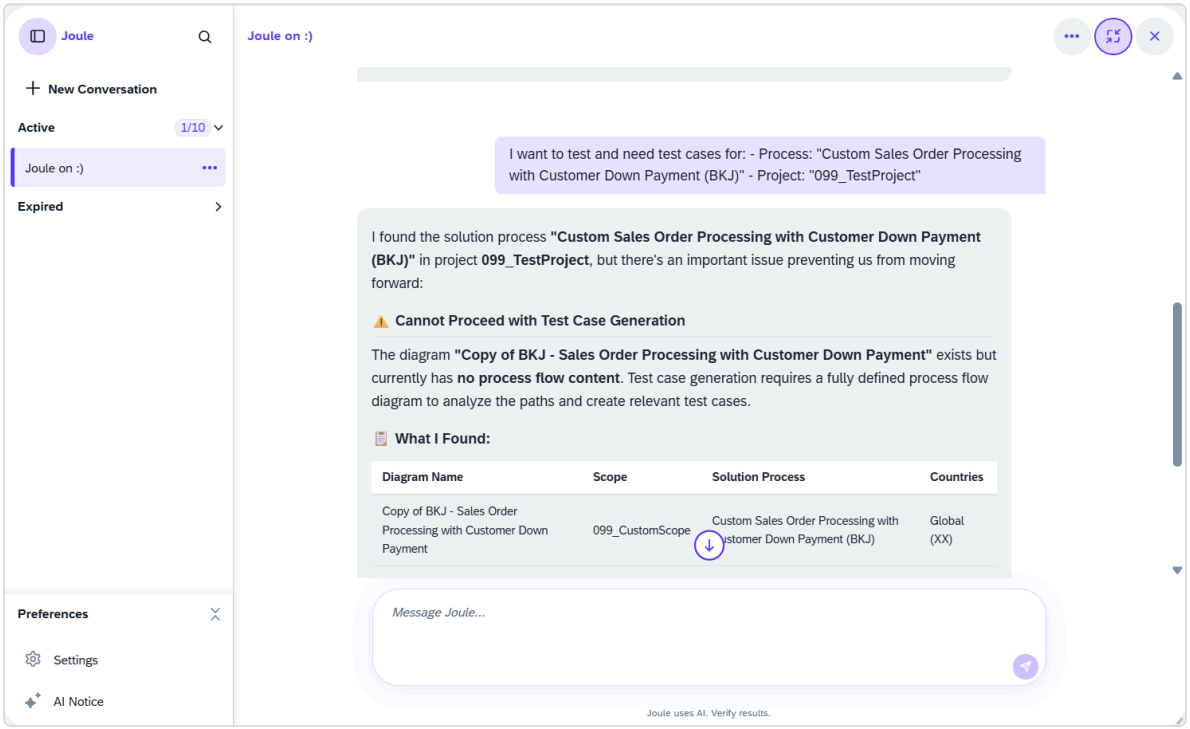
Let's see what Joule does when asked to work with incomplete input.

1. In Joule, prompt it to generate test cases directly; specifying the process and project:

> *"I want to test and need test cases for: Process "Custom Sales Order Processing with Customer Down Payment (BKJ)", Project "XXX_TestProject""*
>
> Replace **XXX** with your workstation number.

Joule navigates to the process in your project and analyses its content, but it hits a problem.

> **Tip:** Click the **expand icon** (⤢) in the Joule panel header to open Joule in full-screen mode: much easier to read Joule's responses, especially for longer agent outputs. The left sidebar shows your conversation list: up to **10 concurrent conversations** can be active at the same time, with older ones moved to **Expired**.

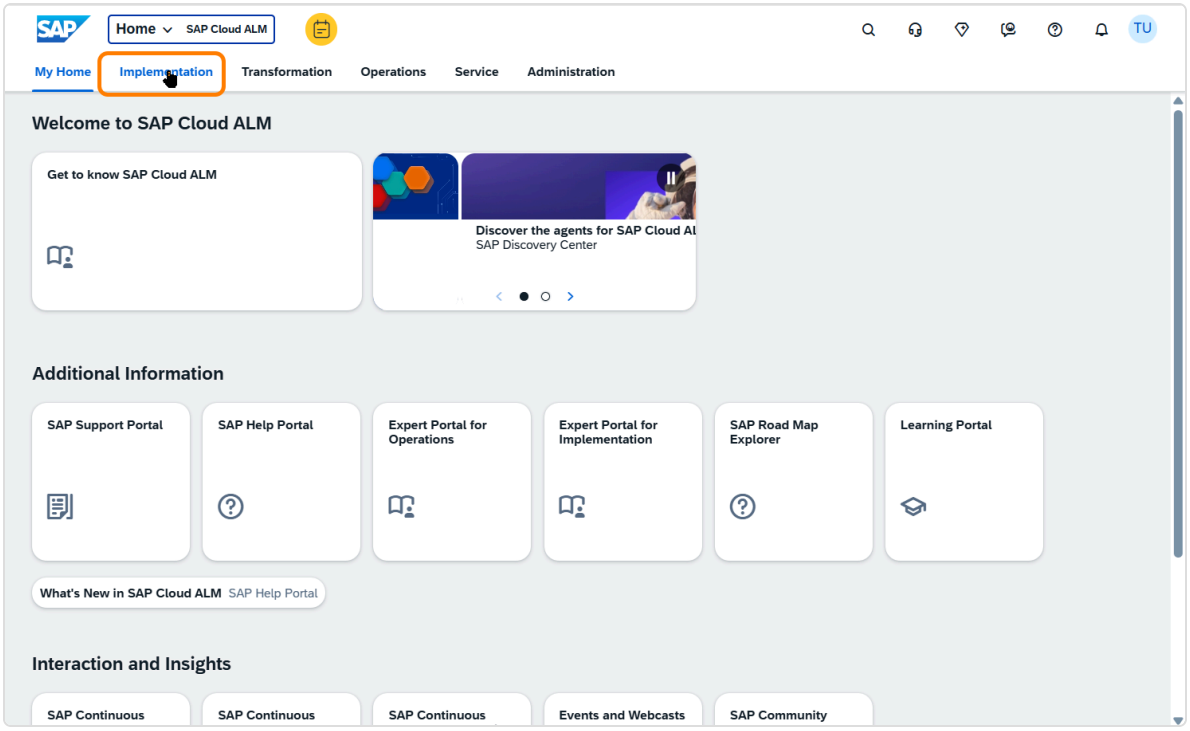


> ⚠️ **Cannot Proceed with Test Case Generation**
>
> Joule found the process but immediately flags that the information in it is insufficient for a meaningful scoping and test case run; the process diagram exists but has no content (no flows, no steps, no acceptance criteria to work from).
>
> This is intentional. **Garbage in, garbage out**: even for AI agents. The quality of the output depends directly on the quality of the input. Before Joule can do useful work, we need to enrich the process with proper documentation.
>
> That's exactly what we'll do in the next steps.

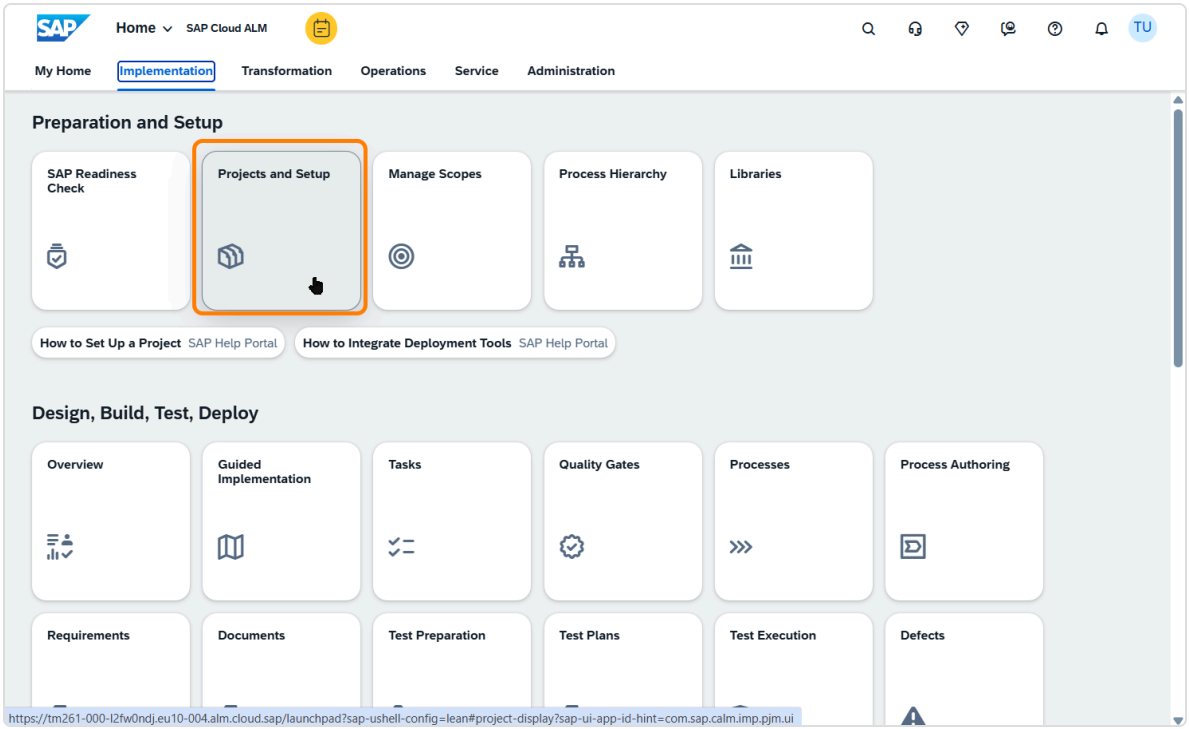
Step 4 – Navigate to Your Project

Time to find our process. Close the Joule panel first (click × in the panel header), then navigate to the project.

1. Back on the home screen, click the **Implementation** tab in the top navigation bar. This is the launchpad for all test management related apps, and much more: requirements, process authoring, test preparation, test plans, test execution, defects.

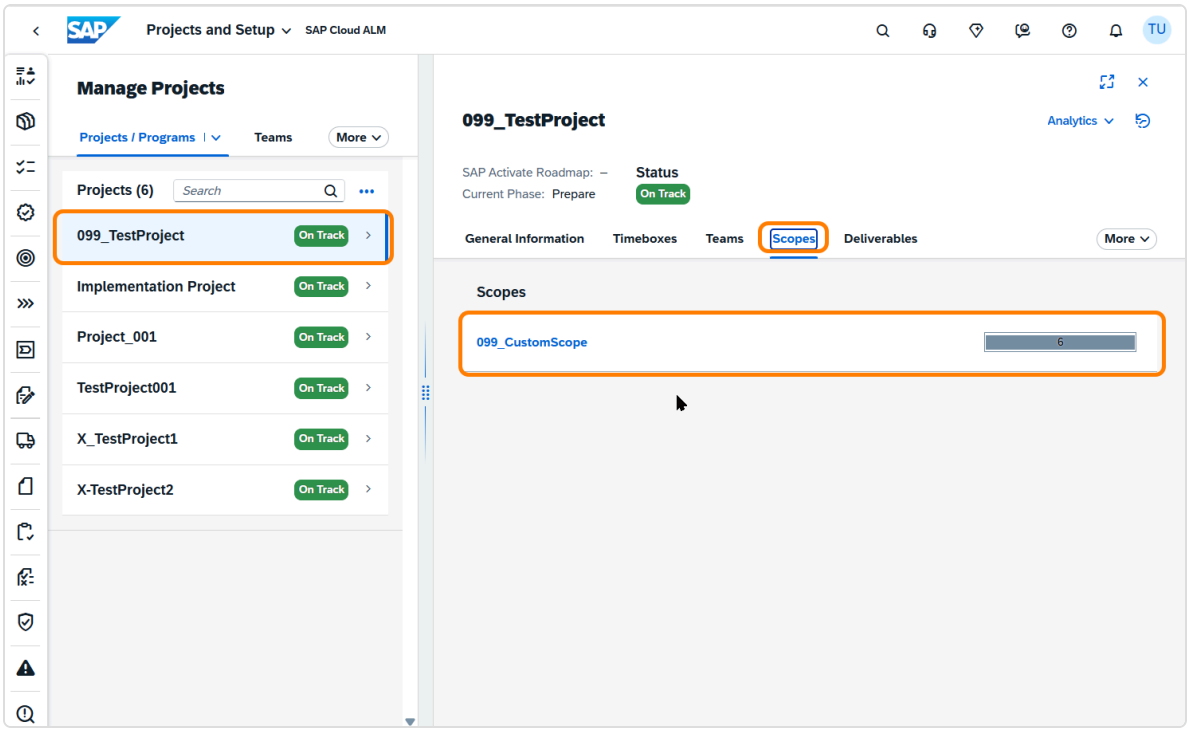


1. Under **Preparation and Setup**, click the **Projects and Setup** tile.



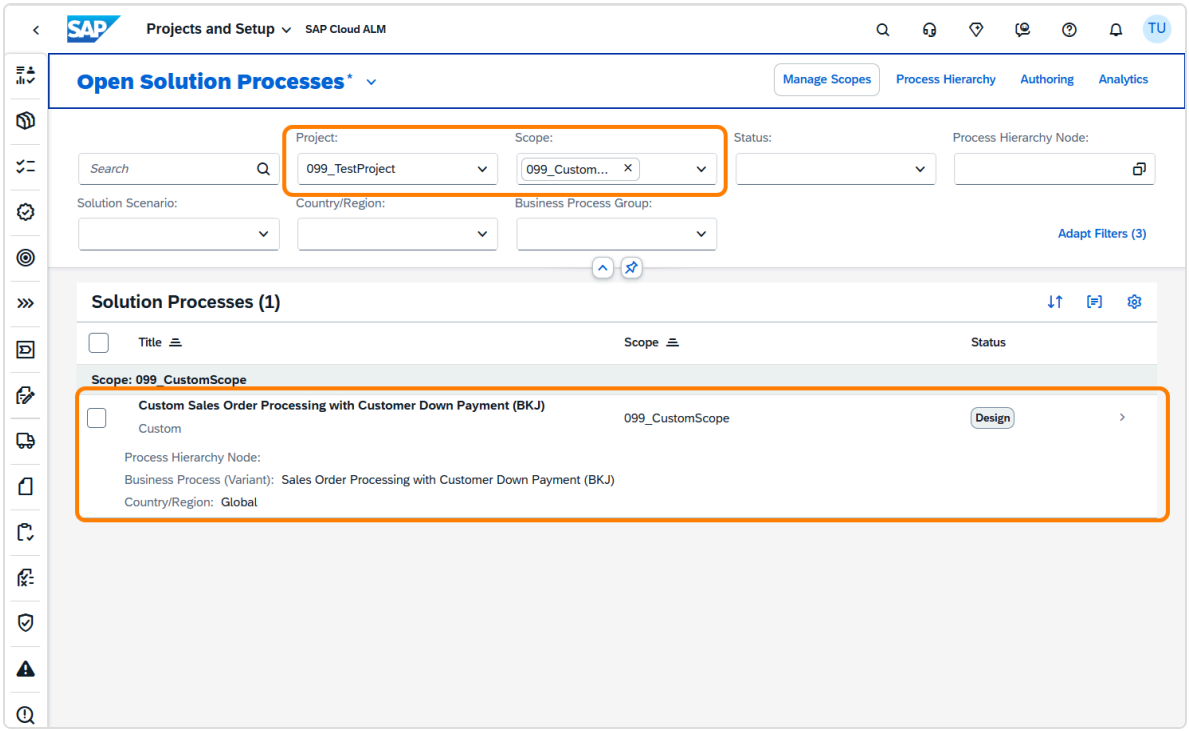
1. In the **Manage Projects** list, locate your project: `XXX_TestProject` (replace XXX with your workstation number). Click it to open the project details.

2. In the project detail view, click the **Scopes** tab. You'll see one scope: `XXX_CustomScope`: this is the container for all the process content you'll work with today. Click on it.



1. The view switches to **Open Solution Processes**, with the filter pre-set to your project and scope. You should see exactly one process: `Custom Sales Order Processing with`

`Customer Down Payment (BKJ)` : your process for today. Click on it to open its details.



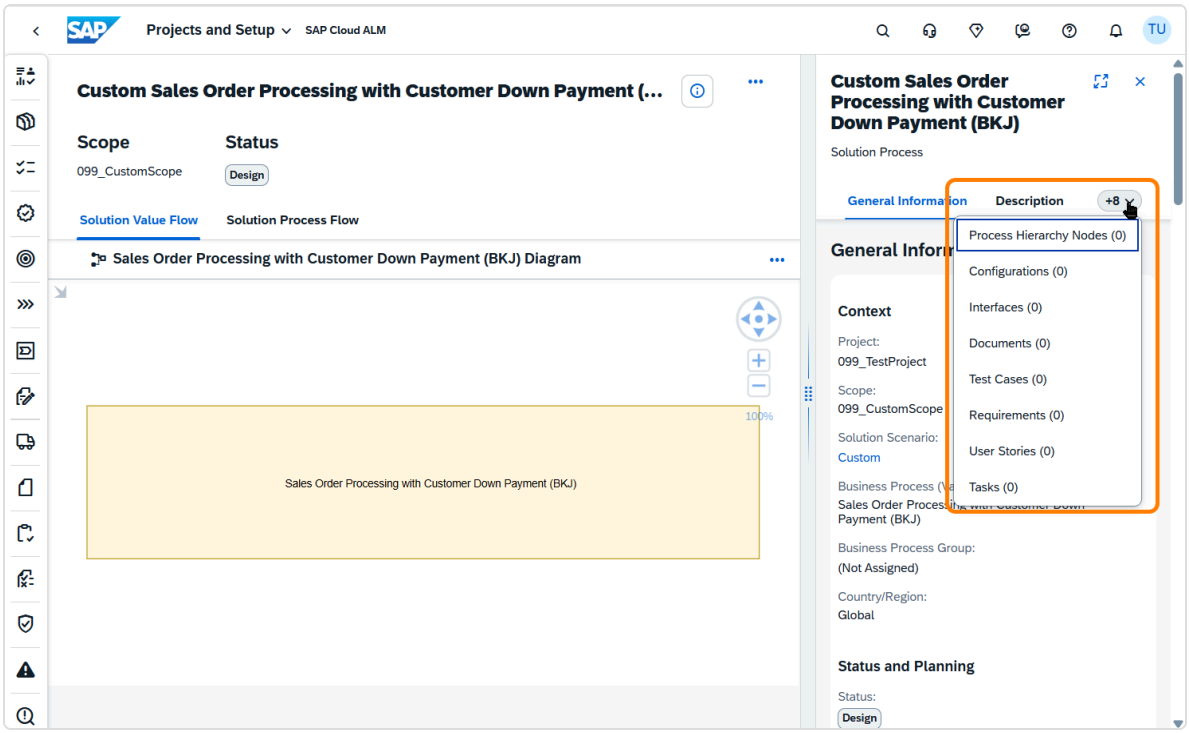
1. The process detail view opens. On the right-hand panel, next to the **General Information** and **Description** tabs, click **+8** to expand the full list of content types that can be linked to this process:

Content Type	Count
Process Hierarchy Nodes	0
Configurations	0
Interfaces	0
Documents	0
Test Cases	0
Requirements	0
User Stories	0
Tasks	0

Everything is at zero. The process diagram on the left is also empty; no flows, no steps. This is your starting point.

> **This is the root cause** of Joule's rejection in Step 3. With no requirements, no user stories, and no process content, there is nothing for the AI agents to work from.

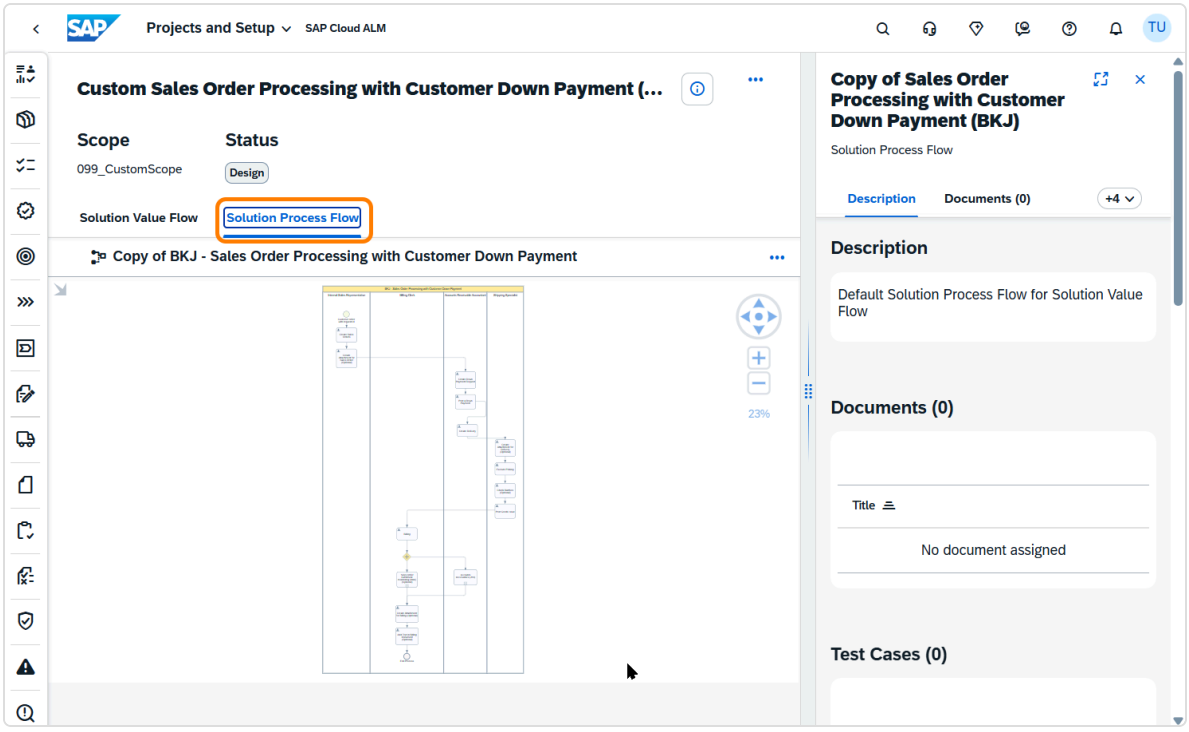
> Before we can generate anything meaningful, we need to populate this process, and that's exactly what the next exercises will do.



1. Now click the **Solution Process Flow** tab (next to Solution Value Flow). Here you'll find something more promising: a BPMN process diagram showing the full BKJ flow across four swimlanes; Internal Sales Representative, Billing Clerk, Accounts Receivable Accountant, and Shipping Specialist.

This diagram was imported from **SAP Signavio** into SAP Cloud ALM. It is natively stored here; a direct live integration with Signavio is planned as a future capability.

> **But here's the catch:** the diagram alone is not sufficient for the AI agents to generate high-quality scope, test plan, and test cases. It provides the process skeleton, but without requirements, user stories, and linked documentation, the agents can't derive meaningful test coverage from it.
>
> Feel free to explore the diagram. You can even make small changes, but don't alter it too dramatically, or the AI agents may flag inconsistencies between the diagram and the other content you'll add next.

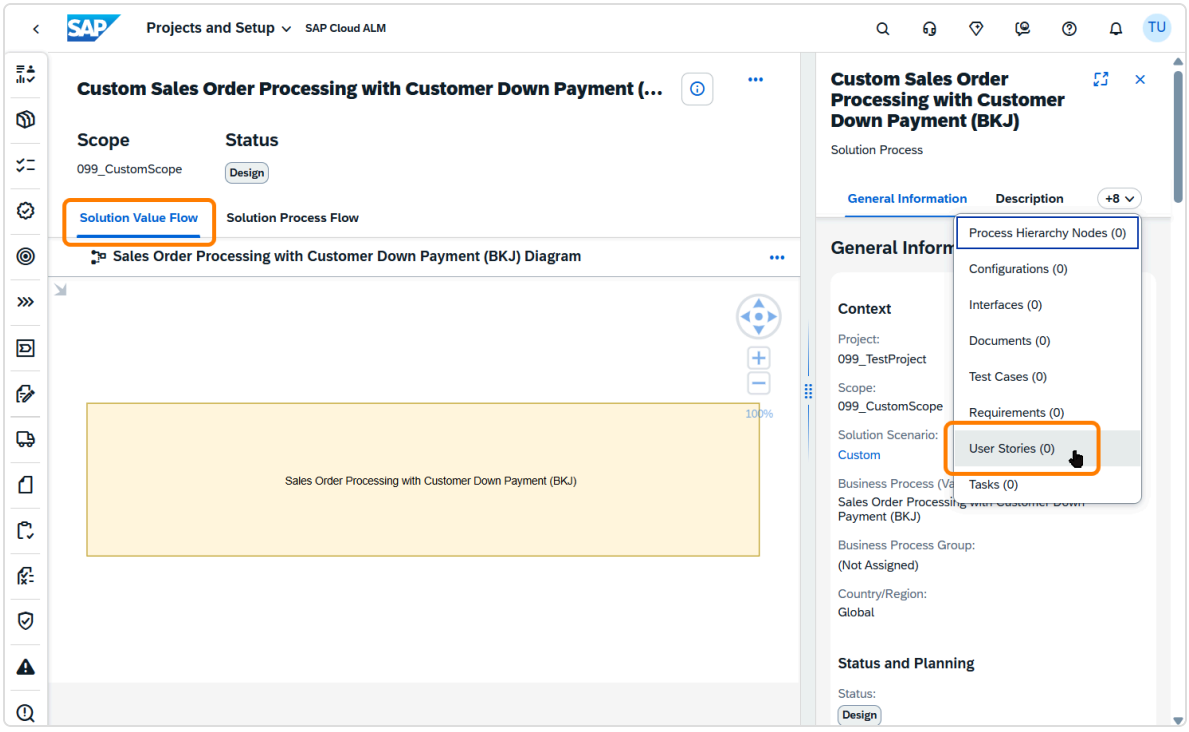


What do you see? A process structure exists, but the process detail panel (User Stories, Requirements, Documents) is still mostly empty. The diagram provides the skeleton; what's missing is the enriched content the AI agents need to do useful work. That's exactly what the next steps will fix.

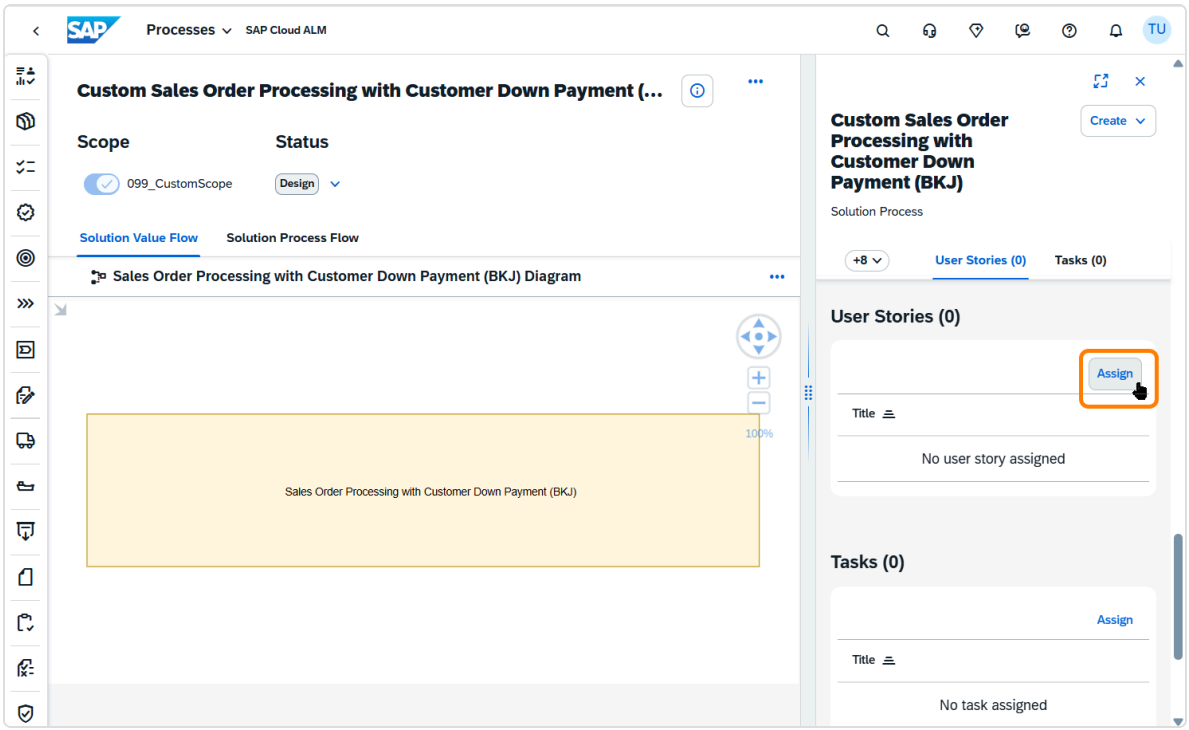
Step 4C – Assign Pre-Prepared User Stories

To give the AI agents enough to work with, we need to link supporting content to the process. Let's start with the user stories.

1. Make sure you are on the **Solution Value Flow** tab. On the right-hand panel, click **+8** to open the dropdown and select **User Stories**: this brings the User Stories section into view.



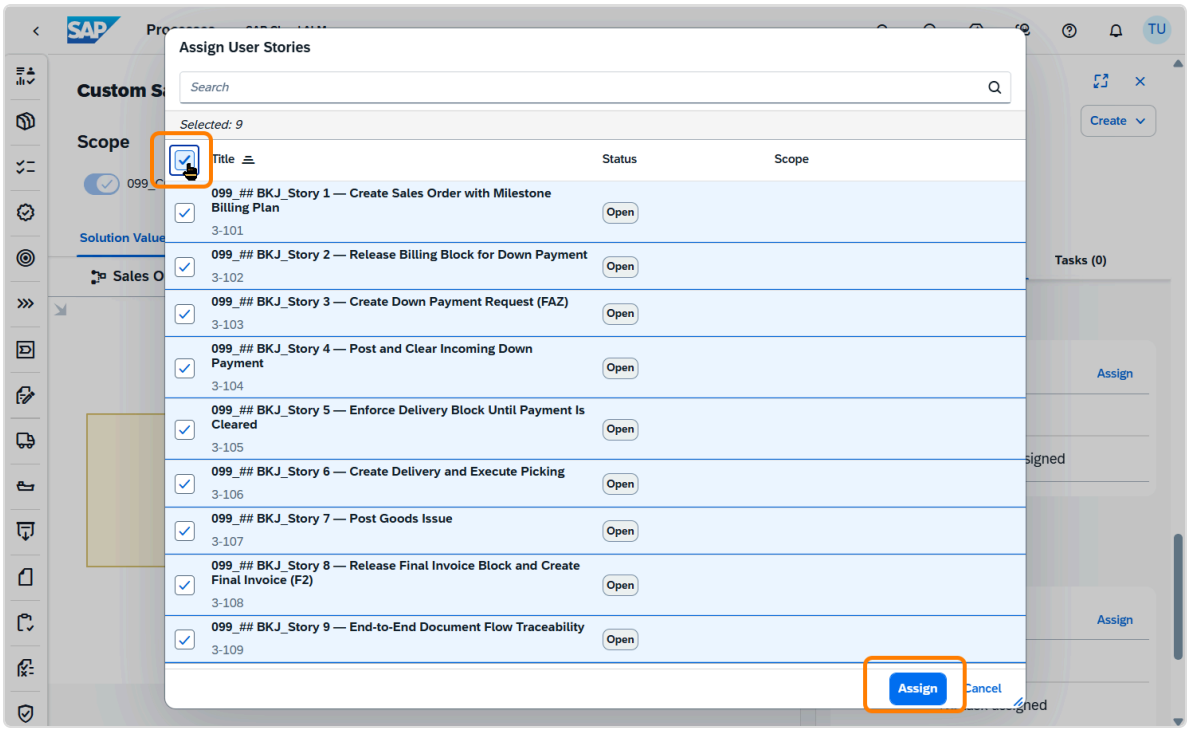
1. The **User Stories** section is now visible in the right panel: currently empty ("No user story assigned"). Click the **Assign** button to open the selection dialog. It comes pre-populated with user stories prepared for the BKJ process.



1. The **Assign User Stories** dialog opens, pre-populated with all 9 BKJ user stories prepared for this exercise:

#	Story
1	Create Sales Order with Milestone Billing Plan
2	Release Billing Block for Down Payment
3	Create Down Payment Request (FAZ)
4	Post and Clear Incoming Down Payment
5	Enforce Delivery Block Until Payment Is Cleared
6	Create Delivery and Execute Picking
7	Post Goods Issue
8	Release Final Invoice Block and Create Final Invoice (F2)
9	End-to-End Document Flow Traceability

Click the **select all** checkbox in the header row (all 9 stories should now show as selected), then click **Assign**.



1. After returning to the process, feel free to open one of the user stories to inspect it. Each story is fully defined with **story points**, the **user story** narrative ("As an [role], I want to... so that..."), and granular **acceptance criteria**: exactly the kind of structured input the AI agents need.

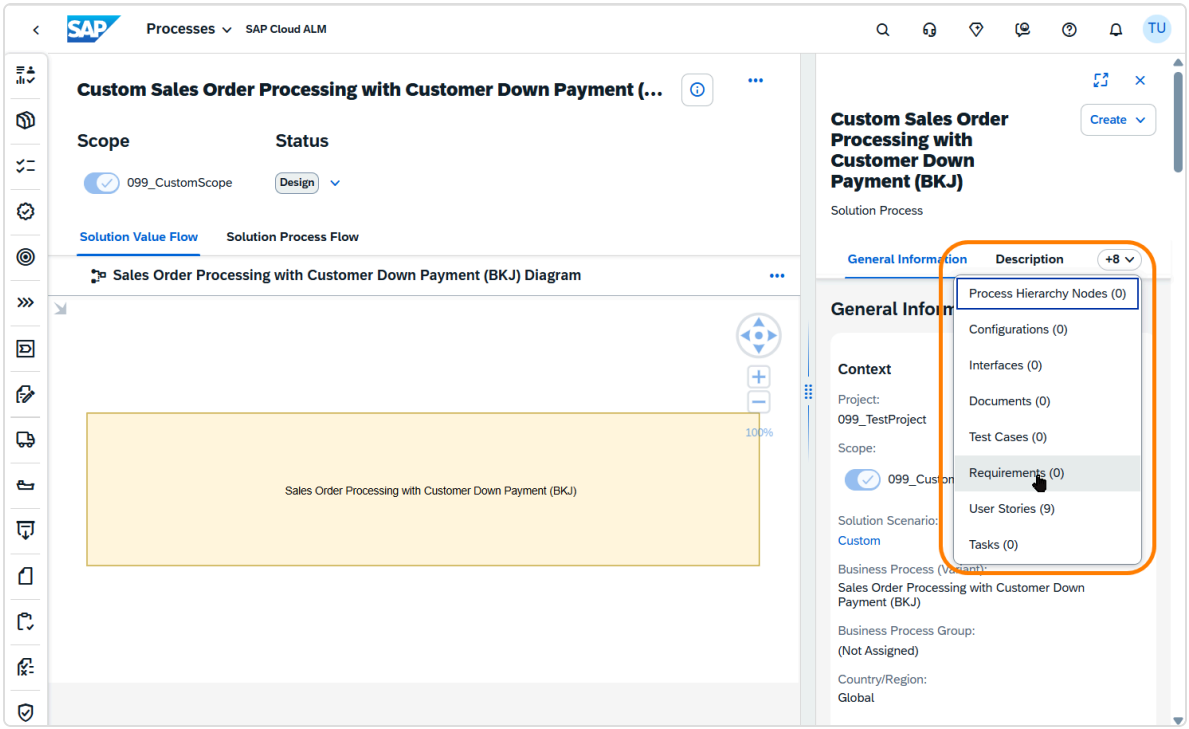
▶ 📷 Example: User Story 1: Create Sales Order with Milestone Billing Plan

When done, click the **back arrow** (←) at the top left of the screen to return to the process view.

1. Back in the process, open the **+8** dropdown again; **User Stories** now shows **(9)**. The assignment worked.

But look at **Requirements (0)**. That entry is still empty, and it's arguably the most important input for the AI agents. A well-formed requirements document gives the agents the business rules, constraints, and acceptance detail they need to generate meaningful test coverage.

Here's the good news: you were recently in a structured interview with a UX colleague, walking through the BKJ process step by step in detail. Your colleague promised to post the transcript in SAP Cloud ALM for further use. Time to retrieve it and turn it into a requirements document.



Summary

You've logged in, met Joule, and oriented yourself in the project:

- ✅ Joule opened and introduced its capabilities
- ✅ Process located: BKJ in `XXX_TestProject / XXX_CustomScope`
- ✅ 9 user stories assigned to the process
- ❌ Requirements document: still missing; that's where we go next

Continue to Exercise 1 – Generate Requirements with AI

Exercise 1 – Generate Requirements with AI

Prerequisite: You have completed Getting Started; the BKJ process is set up with user stories and solution documentation.

What You'll Do

In this exercise you'll use the SAP Cloud ALM **Requirements** app to generate a requirements document from an interview transcript. This is your first AI-assisted step; instead of drafting requirements manually, the AI reads an expert interview and structures it for you.

> **Why this matters:** Good requirements are the foundation for good test cases. Without them, the scoping and planning agents in later exercises won't have enough context to produce high-quality output. You'll see this contrast directly.

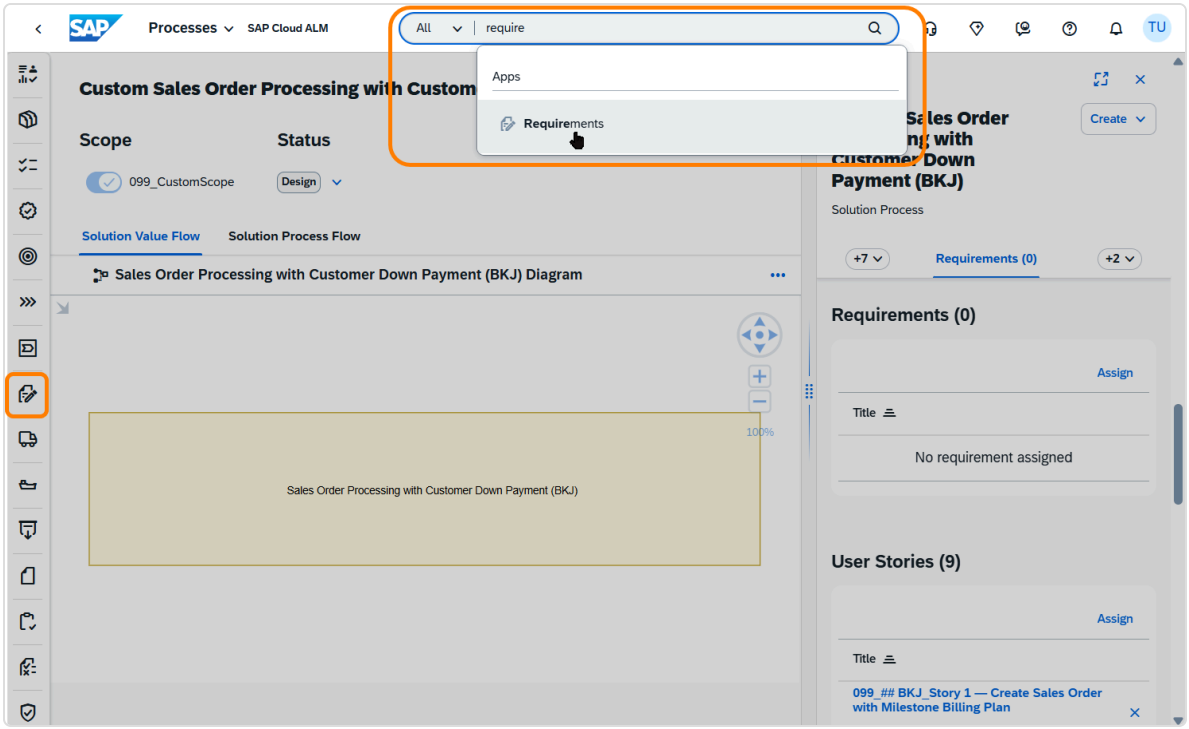
Step 5 – Generate Requirements from an Interview Transcript

A requirements interview with a BKJ process expert was recorded and transcribed for you. The transcript captures all the key business rules, process steps, and constraints discussed in the expert session.

5.1 – Open the Requirements App

Navigate to the **Requirements** app. There are two ways:

- **Search bar** (top center): type `require` and select **Requirements** from the results
- **Left toolbar**: click the quick-link icon that looks like a document being written on with a pen



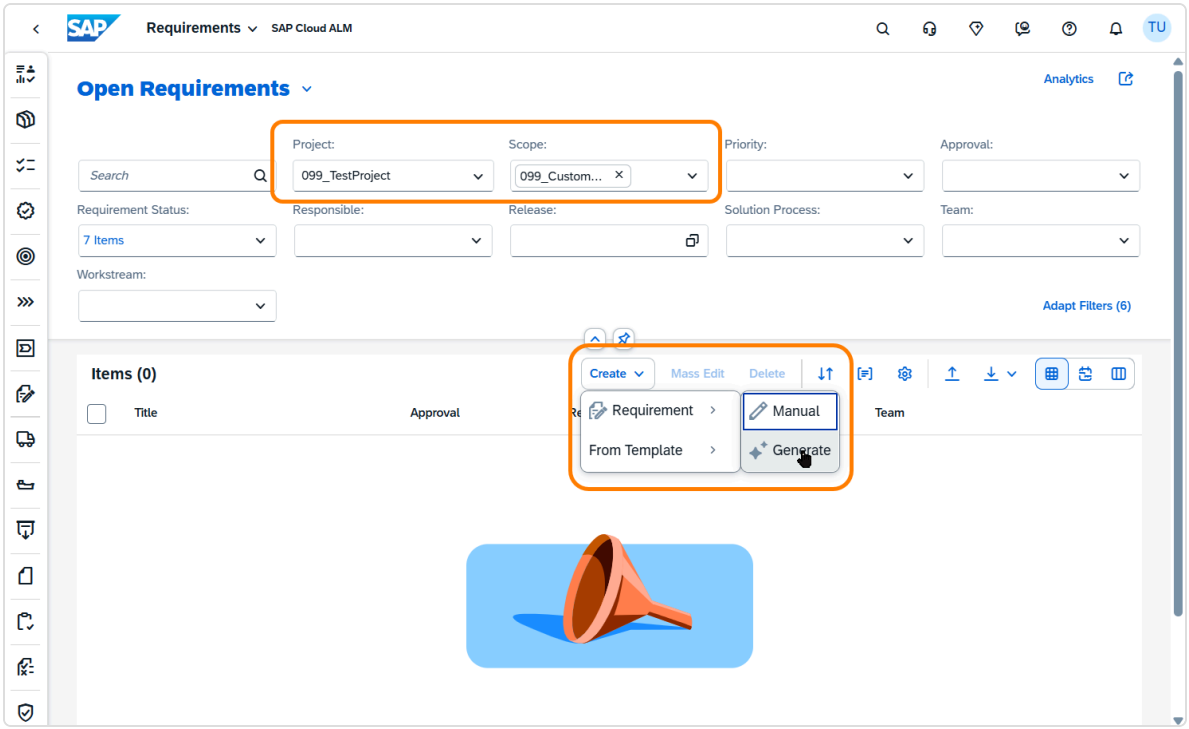
5.2 – Create a Requirement via Generate

1. The Requirements app opens with the filter pre-set to your project and scope; confirm both show your **workstation number** (`XXX_TestProject` / `XXX_CustomScope`). The list is empty: no requirements exist yet.

2. Click **Create**, then hover over **Requirement**: a sub-menu appears with two options:

3. **Manual**: write the requirement yourself
4. **Generate** ✨; let AI generate it from a source document

Click **Generate**.

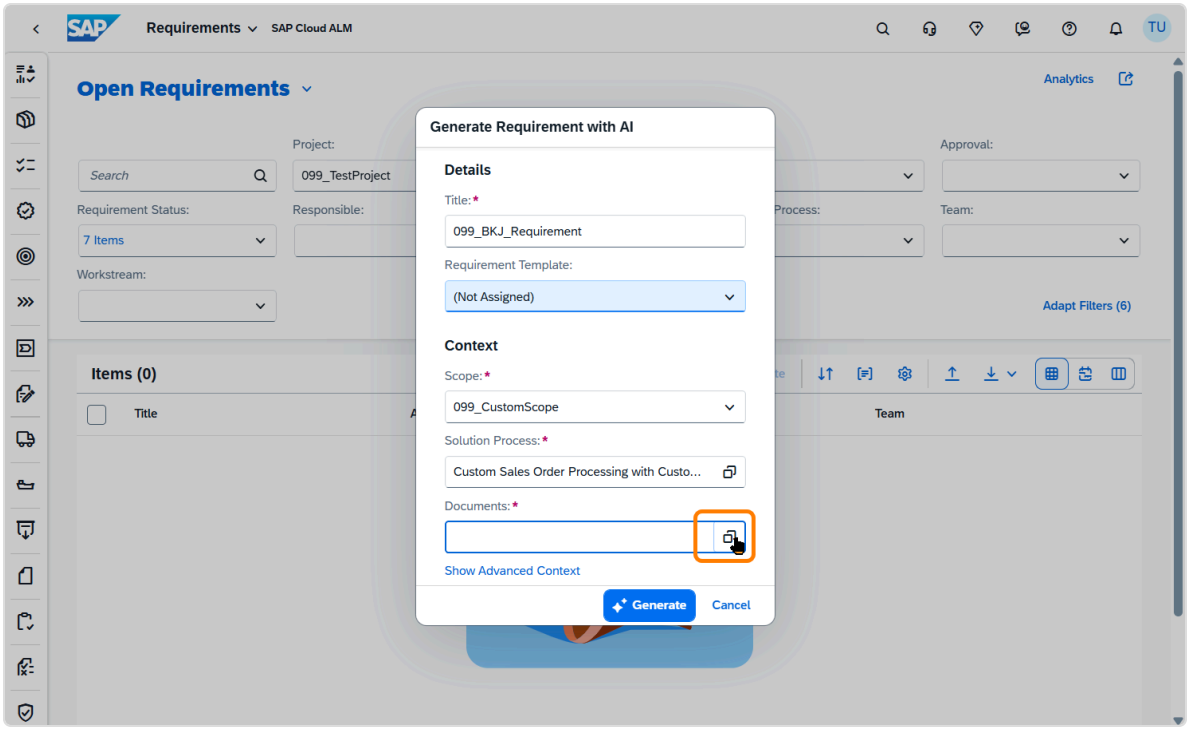


5.3 – Configure the Generation Dialog

The **Generate Requirement with AI** dialog opens. Fill in the fields:

Field	Value
Title	`XXX_BKJ_Requirement` (replace XXX with your workstation number)
Requirement Template	(Not Assigned)
Scope	`XXX_CustomScope`
Solution Process	Custom Sales Order Processing with Customer Down Payment (BKJ)
Documents	← click the selection icon to open the document picker

Click the **selection icon** (📋) next to the **Documents** field to choose the source document; the interview transcript your colleague posted.

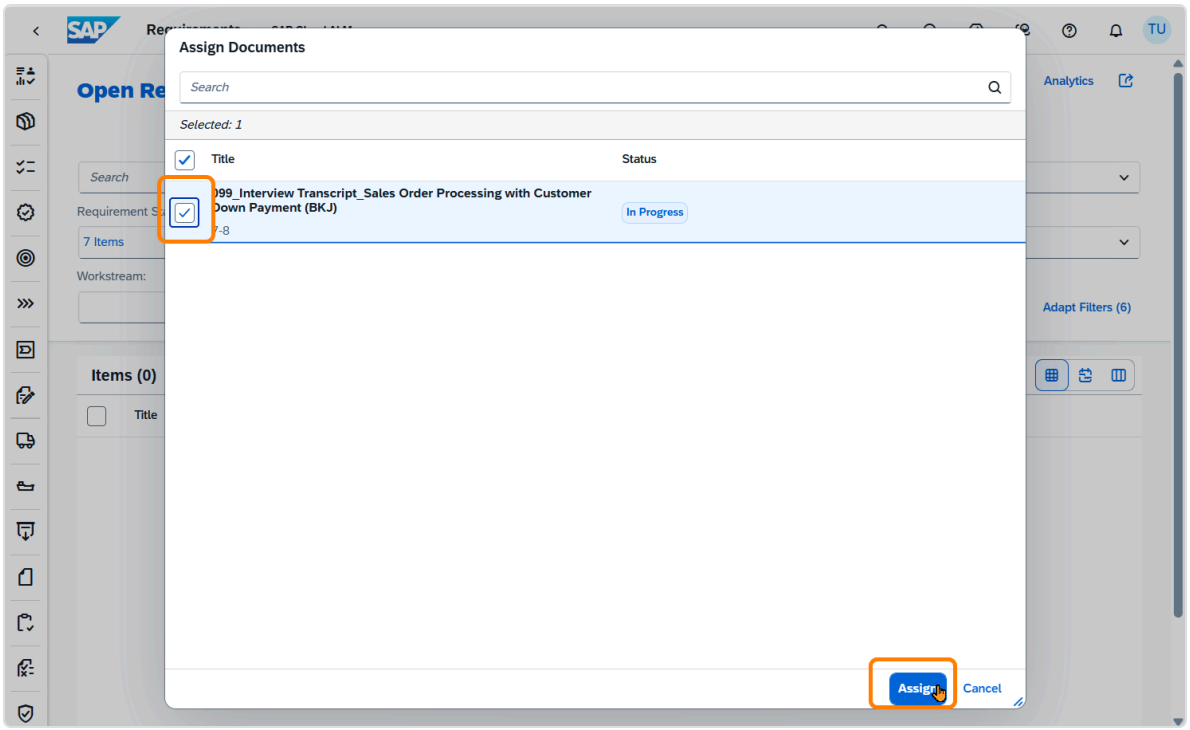


5.4 – Select the Interview Transcript

The **Assign Documents** dialog opens, showing the documents available in your project. You should see the interview transcript your colleague posted:

`XXX_Interview Transcript_Sales Order Processing with Customer Down Payment (BKJ)`

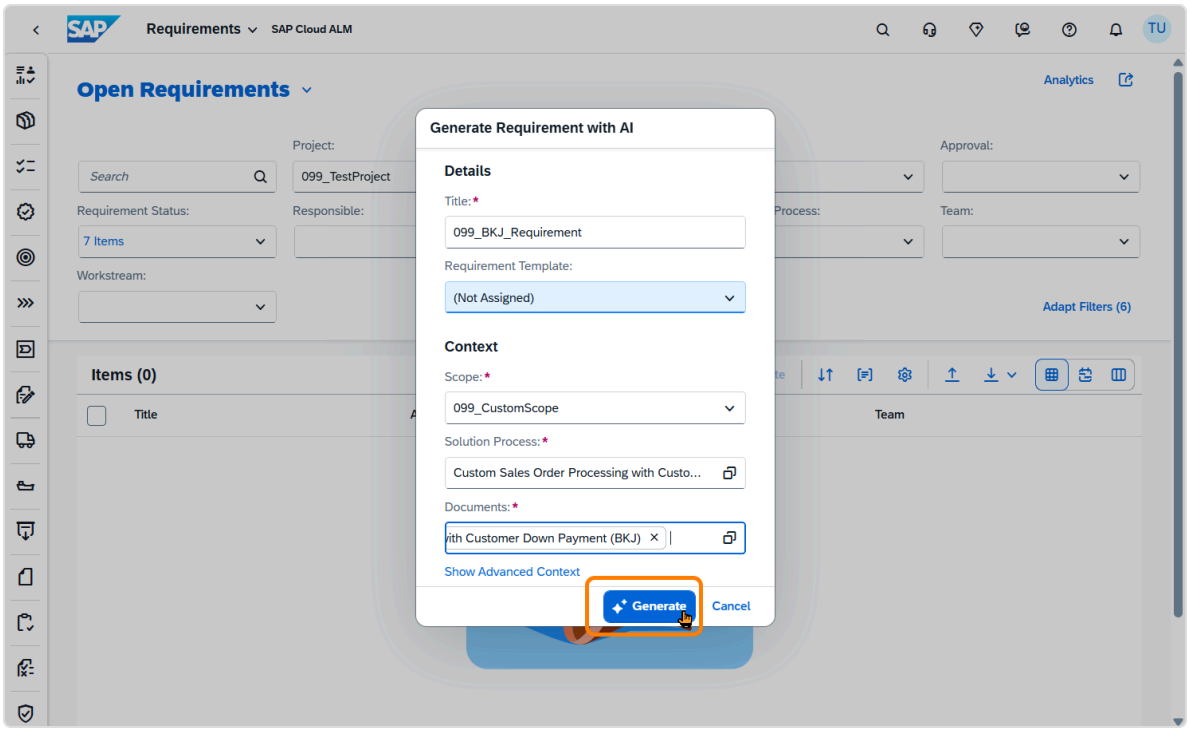
Select it and click **Assign**.



5.5 – Generate

Back in the **Generate Requirement with AI** dialog, the transcript now appears in the **Documents** field. Everything is set. Click **Generate**.

SAP Cloud ALM's built-in AI agent reads the full 30-minute conversation transcript and converts it into a structured requirements document; automatically. You'll be able to review the result in the next screen.



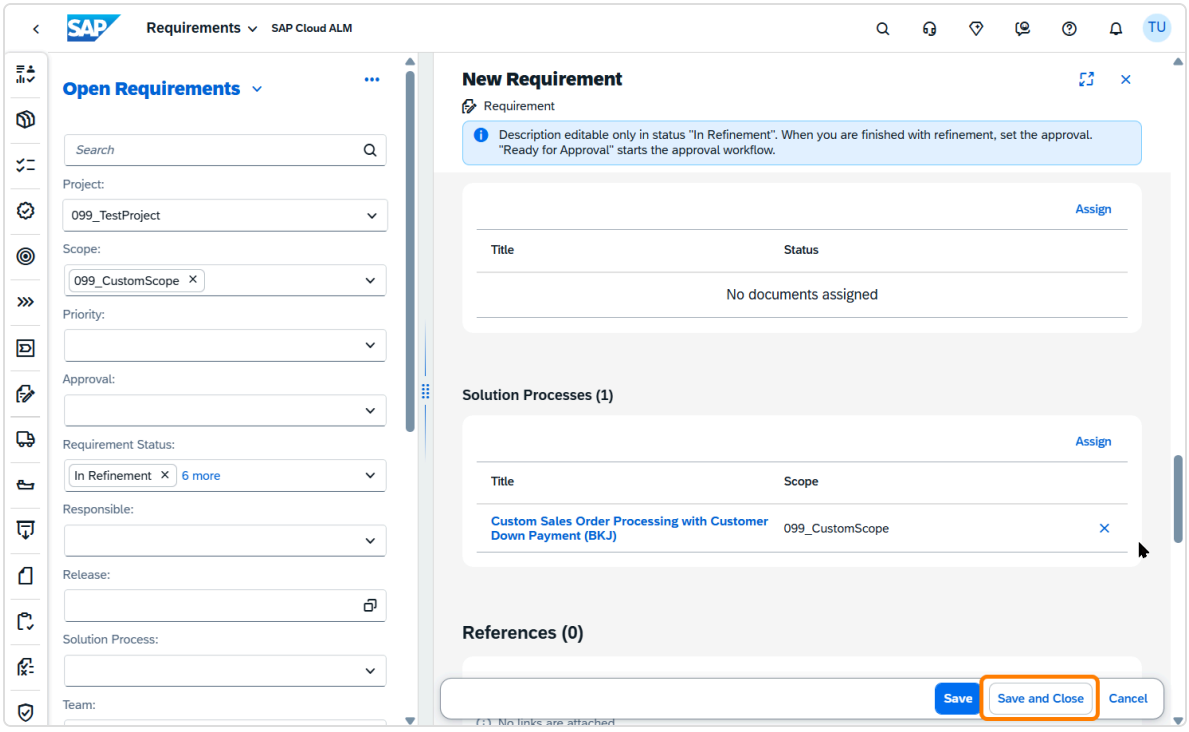
5.6 – Review and Save the Generated Requirement

The AI has created the new requirement. The detail view opens automatically.

A few things worth noting here:

- **Scroll up** to read the generated description; this is the structured requirements content extracted from your interview transcript.
- **Scroll down** to the **Solution Processes** section: the BKJ process has been **automatically detected and linked**: no manual assignment needed. The AI recognised the context from the document.
- The info banner at the top reminds you that the description is editable while the status is *In Refinement*. Feel free to make adjustments if needed.
- At the bottom you can optionally fill in approver, priority, and other fields; skip this for now.

When satisfied, click **Save and Close**.



Summary

Step 5 complete; AI-supported requirements generation done. A 30-minute expert interview was converted into a structured requirements document, automatically linked to the BKJ process.

The process now has everything the AI agents need:
- ✅ BPMN process diagram (from Signavio)
- ✅ 9 user stories (assigned in Getting Started)
- ✅ Requirements document (generated in this exercise)

Time to head back to the process and try what failed in Step 3; this time with proper input in place.

Continue to Exercise 2 – Scope Your Tests with AI

Exercise 2 – Scope Your Tests with AI

Prerequisite: You have completed Exercise 1; the BKJ process has linked requirements, user stories, and solution documentation.

What You'll Do

In this exercise you'll unleash the **Scoping Agent**: an AI agent that reads all your linked process documentation and proposes a structured set of test cases. While the agent works, you'll experience what manual scoping looks like, so you can appreciate the comparison.

Step 6 – Unleash the Scoping Agent

Time to try again; with proper input now in place.

6.1 – Prompt Joule (same as before)

1. Open Joule and send the **same prompt** as in Step 3:

> *"I want to test and need test cases for: Process "Custom Sales Order Processing with Customer Down Payment (BKJ)", Project "XXX_TestProject""*

You can see your newly created requirement in the list on the left; status *In Refinement*, already linked to the BKJ process.



1. The scoping agent starts working in the background. You'll see a notification or progress indicator.



> ⏱ **While the agent works: try it yourself:**
>
> The scoping agent is analyzing your requirements, user stories, solution description, and process structure. This typically takes a few minutes.
>
> In that time, try to do the same thing manually:

1. Look at the BKJ process (9 user stories, the solution steps)
2. **On paper (or in your head):** Try to outline test cases; think about: what apps are involved? What roles? What are the happy path steps? What could go wrong?
3. How many distinct test cases would you define? How long would it take to write them all out properly?

This is what test preparation normally looks like.

Step 7 – What's Happening? (Narration)

Presenter narration: While the agent works, here's what it's doing under the hood:

The scoping agent applies a structured methodology:
1. It reads all linked documentation (requirements, user stories, solution design)
2. It identifies distinct **process paths**: the happy path, role-based variations, error paths
3. It maps each path to **acceptance criteria** from the user stories
4. It groups paths into **test cases** with names, descriptions, and scope assignments

This is the same analysis a skilled test architect would do, but compressed from hours into minutes.

Step 8 – Manual Scoping: Break It Down Yourself

While you wait for the agent, try a structured manual scoping exercise.

For the BKJ process, work through these dimensions on a piece of paper or sticky note:

Dimension	Your Answer
How many distinct **apps/UIs** are involved?	
How many **roles** participate?	
What are the main **process paths** (happy path + variants)?	
Which steps have **blocking conditions** worth testing?	
What **error scenarios** should be covered?	

> Hint: The BKJ process involves at least 4 roles (Internal Sales Rep, Billing Clerk, AR Accountant, Shipping Specialist) and 8 distinct Fiori apps. The process has 2 critical gates: the billing block release and the delivery gate after payment clearance.

Keep your notes; you'll compare them to the agent's output next.

Step 9 – Review the Scoping Agent's Work

The agent is done (you'll see a notification). Let's review what it produced.

9.1 – Open the Agent's Output

1. Go to the **Notification Area** (bell icon) and open the scoping agent's completion notification.
2. This takes you to the proposed test case scope; a list of test cases the agent has defined for the BKJ process.





9.2 – Evaluate the Scope

Review the proposed test cases. For each one, check:

- **Name and description**: does it clearly describe what's being tested?
- **Linked acceptance criteria**: is it traceable back to a user story or requirement?
- **Role**: is the correct role assigned to each test case?
- **Coverage**: does the overall set cover happy path, role boundaries, and the two key gates?



> **Compare with your manual notes:** Did the agent catch everything you identified? Did it find things you missed? This comparison is a key learning moment.

Step 10 – (Optional) Refine the Scope in Joule

The scope is a starting point, not a final decision. Joule lets you refine it through conversation.

Try one of these refinements:

- **Drop an acceptance criterion:** "Remove the batch management acceptance criterion from test case X; it's out of scope for our project."
- **Split a test case by role:** "Split test case Y into two cases; one for the Internal Sales Rep and one for the Billing Clerk."
- **Add coverage:** "Add a negative test case for the delivery block scenario."



Summary

The scoping agent analyzed all your linked process artifacts and proposed a structured test case scope; with traceability to requirements and user stories; in the time it took you to sketch a rough outline manually.

You now have a **confirmed test case scope** ready for test planning.

Continue to Exercise 3 – Plan Your Tests with AI

Exercise 3 – Plan Your Tests with AI

Prerequisite: You have completed Exercise 2; a confirmed test case scope exists for the BKJ process.

What You'll Do

In this exercise you'll unleash the **Planning Agent**: an AI agent that takes your test case scope and builds a structured **test plan**: organized test case entries with descriptions, assigned roles, preconditions, and logical groupings. While it works, you'll compare it with the effort of doing this manually.

Step 11 – Unleash the Planning Agent

11.1 – Confirm the Scope and Prompt for Planning

1. Make sure you're satisfied with the test case scope from Exercise 2. If you made refinements via Joule, confirm the scope is in its final state.
2. Open Joule and enter: **"The scope looks good. Please create a test plan."** or **"Confirm scope, go into planning."**



1. The planning agent queues and starts. You'll see a running indicator.



> ⏱️ **While the agent works: try it yourself:**
>
> Take one of the test cases from the scope and try to manually write a test plan entry:
>
> 1. Pick a test case (e.g. "Create Sales Order with Billing Plan")
> 2. Try to write: a clear description, the role performing it, the preconditions (what must be true before this step), and a rough sequence of test steps
> 3. How long does that take for one test case? Now multiply by the full scope...
>
> This is what test plan preparation normally looks like.

Step 12 – What's Happening? (Narration)

The challenge: Scope decisions rely on expertise that cannot scale; coverage becomes patchy and acceptance criteria unreliable. For each test case a test analyst would normally have to determine which app to use, define preconditions, sequence the steps at the right level of detail, assign the correct role, and link everything back to requirements. Multiply that by a full BKJ scope.

The solution: The **Preparation Agent** maps execution paths from the process model into complete test cases. The expert shifts from producing to reviewing, and covers scope more reliably.

Here is what the agent is doing right now, step by step:

Stage	What happens
Validate incoming scope	Checks quality and completeness of the scope context from the Scoping Agent. Flags issues before test generation begins.
Map all process variants	Reads the BPMN diagram to identify every execution path: happy path, error flows, boundary conditions, auth checks, negative flows.
Draft test plan	Creates a categorised plan covering all identified process variants; this is what you'll review and approve in Step 13.
HITL Gate 2: Approve plan	⛔ **No test cases are written to Cloud ALM without your explicit approval.** You review the plan and adjust before the agent continues.
Write test cases	Creates test cases in Cloud ALM via ContentAssist. Progress is streamed live so you can watch it work.
Link cases and apply EARL	Links each test case to its Requirement and Solution Process. Applies SAP standard content (EARL) where available.
Output	Complete, traceable test cases written to Cloud ALM Test Management; ready for execution planning.

The gate in the middle (HITL Gate 2) is deliberate: the agent cannot proceed past the plan without human sign-off. You are always in control.

Step 13 – Review the Agent's Test Plan

13.1 – Open the Completed Plan

1. Go to the **Notification Area** and open the planning agent's completion notification.

2. Review the generated test plan; you should see test cases organized with descriptions, roles, and preliminary step structure.





13.2 – Inspect a Test Case

1. Open a test case in detail; look at the groupings, preconditions, and step-level descriptions.



> **Traceability check:** Does the test case show its link back to the scope item and the user story? Good test plans maintain this chain; it's essential for coverage reporting and audit.

13.3 – Check Role Transitions

One of the most common planning mistakes is mixing multiple roles within a single test case. Look at the BKJ process; it involves 4 roles with 5 handover points.

> **Look for:** Are there test cases in the plan that should be split by role? For example, is the delivery step (Shipping Specialist) properly separated from the billing step (Billing Clerk)?



Step 14 – (Optional) Refine the Test Plan in Joule

Use Joule to make targeted adjustments to the plan without restarting from scratch.

Try one of these:

- **Split by role:** "Split test case 'Create and Release Billing Documents' into two; one for the Internal Sales Rep (billing block release) and one for the Billing Clerk (billing document creation)."
- **Add a precondition:** "Add a precondition to the delivery test case: down payment must be cleared before delivery creation can proceed."
- **Drop a step:** "Remove the stock setup step from the main test case; it's covered by a prerequisite setup script."



Summary

The planning agent structured your test case scope into a workable test plan; with roles, preconditions, and step groupings; in minutes.

You now have a **confirmed test plan** ready for test case generation.

Continue to Exercise 4 – Generate Test Cases with AI

Exercise 4 – Generate Test Cases with AI

Prerequisite: You have completed Exercise 3; a confirmed test plan exists for the BKJ process.

What You'll Do

In this exercise you'll unleash the **Test Case Generation Agent**: an AI agent that takes the test plan and writes detailed, executable test cases. While it works, you'll draft one test case yourself, then compare results. After that you'll dive into one of the most overlooked aspects of test case quality: **test data and parameters**.

Step 15 – Unleash the TC Generation Agent

15.1 – Confirm the Plan and Prompt for Generation

1. Make sure your test plan from Exercise 3 is in its final state.
2. Open Joule and enter: **"The plan looks good. Please generate the test cases."** or **"Confirm plan, start test creation."**



1. The generation agent starts. This is the most comprehensive agent; it writes detailed steps, expected results, and parameter placeholders for every test case in the plan.



> ⏱ **While the agent works: try it yourself:**
>
> Pick one test case from your plan (e.g. "Post Incoming Payment and Clear Down Payment") and try to write it manually:
>
> 1. Break it into functional units: what app? what action?
> 2. Assign the correct role: AR Accountant
> 3. Write the test steps at the right level of detail (not too high, not too granular)
> 4. For each step, write the expected result

> **In particular:** note down any input parameters (company code, posting date, customer account, amount, house bank). Draw arrows to show dependencies; which parameter values are determined by earlier steps?
>
> Keep your notes for Step 18.

Step 16 – Manual Test Case Work

While the agent runs, try a more structured breakdown:

For your chosen test case, fill in this template:

Element	Your Draft
Test Case Name	
Role	AR Accountant
Precondition	
App	Post Incoming Payments (F1345)
Step 1	Open the app, select company code
Step 1: Expected Result	
Step 2	Enter posting date and journal entry type DZ
Step 2: Expected Result	
Step 3	Filter for Special G/L transactions, find FAZ open item
Step 3: Expected Result	
Step 4	Select FAZ item, choose Clear, Post
Step 4: Expected Result	
Parameters needed	Company code, posting date, G/L account, house bank, customer, amount

---

Step 17 – Review the Generated Test Case

The agent has finished. Let's see what it produced.

17.1 – Open the First Test Case

1. Go to the **Notification Area** and open the TC generation agent's completion notification.



1. Navigate to the first generated test case and open it in detail.





17.2 – Quality Check

Review the generated test case against these criteria:

Quality Criterion	Check
Correct role assigned	☐
Clear, actionable steps (not too vague, not too granular)	☐
Expected result for every step	☐
Input parameters identified	☐
Traceable to a user story or requirement	☐
Logical grouping of steps	☐

> **Compare with your manual draft:** Did the agent's version match what you wrote? What did it do better? What would you change?

Step 18 – Understand Test Data Parameters

This is where test preparation often gets underestimated. Let's go deeper.

18.1 – Identify the Parameters

Open the test case you've been looking at (the payment posting case, or any other complex one) and focus on the **input parameters**: the specific values that must be entered

to execute each step.

1. For each step in the test case, note down all parameter fields (not the values yet, just the parameter names).



1. Now think about **dependencies** between parameters. For the BKJ process:
2. The **Sales Organization** (e.g. 1010) determines which **Distribution Channel** and **Division** are available
3. The **Sold-to Party** (customer) determines the **Ship-to Party** and **Payer**
4. The **Plant** (e.g. 1010) determines which **Storage Location** is available
5. The **Material** type determines which **Item Category** is auto-assigned (CBAO, CTAD, or CTAX)

Draw or describe these dependency relationships; which parameter determines which others?



> **This dependency map is what the Data Agent will use in Exercise 5**: it's not just about picking values, it's about picking consistent combinations.

18.2 – Note the Test Case ID

1. Note down the **test case ID** of the case you've been reviewing. You'll use it to prompt the data agent in Exercise 5.



Summary

The TC generation agent wrote detailed, executable test cases for the full BKJ scope; complete with steps, expected results, and parameter placeholders.

You now understand:
- What a high-quality AI-generated test case looks like

- How to evaluate it for correctness, completeness, and traceability
- That **test data and parameters** are a critical dimension that needs its own attention

Continue to Exercise 5 – Generate Test Data with AI

Exercise 5 – Generate Test Data with AI

Prerequisite: You have completed Exercise 4; generated test cases exist and you have noted a test case ID.

What You'll Do

In this exercise you'll unleash the **Data Agent**: an AI agent that analyzes your test cases and proposes meaningful test data: parameter values with the right combinations, boundary cases, and reasoning for each choice. This is the final AI-assisted stage of test preparation.

Step 19 – Unleash the Data Agent

19.1 – Prompt Joule with Your Test Case

1. Open Joule.
2. Enter a prompt that references the test case ID you noted in Exercise 4. Something like: **"Please suggest test data for test case [your-TC-ID]."**



1. The data agent queues and starts analyzing the test case; reading its steps, identifying parameters, and cross-referencing the master data requirements from the solution design.



> ⏱ **While the agent works:**
>
> Think about what makes good test data:
> - Values that are **valid but exercise boundaries** (e.g. a down payment that is exactly 50% of the order value)
> - Values that are **invalid** but in realistic ways (e.g. insufficient stock, wrong billing type)
> - Values that **respect dependencies** (sales org determines customer pool, plant determines storage)

> What would you suggest as test data for the payment posting test case?

Step 20 – What's Happening? (Narration)

> **The challenge:** Finding valid test data demands expert knowledge that doesn't scale; wrong data corrupts test results and steals time from actual testing.
>
> **The solution:** The **Test Data Management Agent** identifies every required data parameter from test case instructions; the tester looks up the known targets; no expert required. *(This is Phase 1 of a longer roadmap.)*
>
> Here is what the agent is working through right now:

Stage	What happens
Fetch test case	Reads the test case and all its activity steps from Cloud ALM via MCP (TL102). ⚠️ If the test case is not found, has no activities, or the text is empty; the agent stops and asks you to verify the input.
Extract parameters	Identifies every required SAP business object and field value from the instruction text; every line of "Enter Sales Organization..." becomes a structured parameter.
Classify parameters	Labels each parameter by category: **Org Data · Master Data · Config Data · Transactional Data**.
Apply project context	Obtains fixed values (company code, plant, client) from the Cloud ALM project record; so you don't have to look those up yourself.
Output	A read-only parameter table per category. Structured guidance for the tester who locates the actual system values manually.

> **Phase 1 scope note:** The agent identifies and classifies parameters; it does not yet automatically look up system values. That's the tester's final step: use the table as a checklist to find values in the target system.

Step 21 – Review the Test Data Agent Output

21.1 – Open the Suggested Data

1. Go to the **Notification Area** and open the data agent's completion.



1. Review the suggested test data; you should see a structured set of parameter values with explanations.



21.2 – Evaluate the Data Quality

For each parameter group, check:

Aspect	What to look for
Completeness	Are all required parameters covered? Any missing?
Consistency	Are the values consistent with each other (e.g. correct sales org + customer combination)?
Coverage	Does the set include both valid (happy path) and invalid (negative test) values?
Reasoning	Does the agent explain *why* specific values were chosen?
Boundary cases	Are there values that test edge conditions (max amount, exact threshold)?



Step 22 – Go Deeper into Data Understanding

22.1 – Explore Dependencies

1. In Joule, ask: **"What are the dependencies between these parameters?"** or **"Which parameters drive the values of other parameters?"**

The agent should outline the dependency chain; for example, that the Sales Organization determines which customers and materials are valid.



> **Key insight:** The dependency map the agent produces here is exactly what you started sketching in Exercise 4, Step 18. Notice how parameters like Sales

> Organization (1010), Plant (1010), and Material type cascade into choices for all other fields.

22.2 – Reference Data in Context

For the BKJ process, the core master data values are:

Parameter	Value (Demo System)	Determines
Sales Organization	1010	Dist. Channel, Division, Customer pool
Distribution Channel	10 (Direct Sales)	Available customers
Division	00	–
Sold-to Party	10100004	Ship-to, Payer
Plant	1010	Storage location, available materials
Storage Location	101A	–
Material	TG11 / TG12	Item Category (CBAO)
Item Category	CBAO	Billing plan type
Shipping Point	1010	Delivery creation
House Bank	DEBK1 / DEAC1	Payment posting

> **Additional scenario:** The agent's real value shows when you have **unexpected combinations**: e.g. what happens if a non-stock material (NS0002) is used instead of a trading good? The item category changes to CTAX, which uses a different billing plan configuration. These scenarios are hard to spot manually but the agent flags them as meaningful test variants.



Step 23 – Recap: Value Created

Let's step back and look at what you've accomplished in this hands-on.

Time comparison

Stage	Manual Effort (estimate)	AI-Assisted
Requirements from transcript	2–4 hours (BA writing)	~2 minutes (generation) + review
Test case scoping	4–8 hours (architect)	~3 minutes (agent) + review
Test plan creation	2–4 hours (test analyst)	~3 minutes (agent) + review
Test case writing	1–2 days (test analyst)	~5 minutes (agent) + review
Test data suggestion	4–8 hours (analyst + BA)	~2 minutes (agent) + review
Total	**3–5 days**	**< 30 minutes**



> **The shift:** You didn't just save time; you changed the nature of the work. Instead of *creating* artifacts from scratch, you spent your time *reviewing and validating* AI-generated artifacts. That's the role of the future test expert: quality judgment, not manual production.

Summary

The data agent provided meaningful, dependency-aware test data suggestions with reasoning; turning a multi-hour analysis task into a review exercise.

You've now completed the full AI-assisted test preparation cycle:
1. ✅ Requirements generated from interview transcript
2. ✅ Test case scope proposed by scoping agent
3. ✅ Test plan structured by planning agent
4. ✅ Test cases written by TC generation agent
5. ✅ Test data suggested by data agent

Continue to Exercise 6 – Recap & What's Next

Exercise 6 – Recap & What's Next

Prerequisite: You have completed Exercises 0–5.

Step 24 – Recap: A Day in the Life

You just experienced a complete AI-assisted test preparation workflow; from zero context to ready-to-execute test cases with test data; for a real, complex SAP S/4HANA business process.

Here's the journey you took:

```
Day starts → Process assigned (BKJ)
     │
     ├─ Requirements generated from expert interview transcript
     │     └─ No BA time, no writing; AI reads and structures
     │
     ├─ Scoping agent analyzes process artifacts
     │     └─ Produces test case scope with traceability
     │
     ├─ Planning agent structures the scope
     │     └─ Test plan with roles, preconditions, groupings
     │
     ├─ TC generation agent writes the test cases
     │     └─ Executable steps with expected results
     │
     └─ Data agent suggests test data
           └─ Parameter values, dependencies, variants
```

What used to take 3–5 days now takes < 30 minutes of AI-driven generation + your expert review.

The key shift: you moved from *producing* test artifacts to *judging* them. Your expertise is applied where it matters most; validating quality, not filling templates.

Step 25 – Outlook: What's Coming Next

What You Saw Today Is Just the Beginning

The AI capabilities you worked with are the first wave. Here's where SAP Cloud ALM Test Management is heading:



Wave 2: Coming enhancements:

Capability	What it means
Test execution assistance	AI guides the tester through steps in real time, compares system state to expected results
Defect auto-classification	AI analyzes failures and suggests root cause categories
Coverage gap detection	AI identifies process areas with insufficient test coverage
Cross-scope impact analysis	AI flags when a scope item change affects existing test cases
EARL knowledge integration	Deeper use of SAP Best Practice content (EARL) for more precise step generation

> **The bigger picture:** SAP Cloud ALM is building toward a **360° test expert role**: someone who orchestrates AI agents across the full test lifecycle rather than manually executing each stage. Today you got a preview of that future.

Step 26 – This Is a Beginning, Not the End

Key Takeaways

1. **Input quality drives output quality.** The agents worked well because you provided complete, high-quality input: requirements, user stories, solution description. Garbage in, garbage out; still applies to AI.
2. **Human judgment is the irreplaceable layer.** The agents produced the first draft. You provided the quality review; catching role mismatches, missing preconditions, incorrect scope. That's not going away.
3. **The workflow is the product.** It's not about any single AI feature. It's about the **end-to-end pipeline** from process documentation to executable test cases; with AI at every stage.
4. **Traceability is preserved.** Every generated artifact is linked back through the chain: test data → test case → test plan → scope → user story → requirements → process. This is what makes AI-generated content auditable and trustworthy.

Questions & Discussion

▶ 📷 View session closing screenshot

Some questions to discuss:

1. Where in your current testing process would AI agents save you the most time?
2. What inputs do you have today that you'd want to feed to a scoping agent?
3. What does "human in the loop" mean for your team; at which stage would you want to review and confirm?

Feedback

We'd love to hear from you:
- [Leave feedback via the QR code at the front of the room]
- [Or at the session feedback survey link]

Thank You

TM261: AI-Powered Test Management with SAP Cloud ALM

This material will remain available after the session at the SAP TechEd GitHub repository.
